

Guide de l'authentification

Angular + Laravel + MySQL

Parcours complet : interface, API PHP, securite, base de donnees et retour vers l'interface.

Angular UI	Nginx	Laravel API	MySQL	JSON + cookies
------------	-------	-------------	-------	----------------

Document base sur le code du projet angular-laravel-auth-starter.

1. Vue d'ensemble

Le projet est une application SPA : Angular affiche les pages dans le navigateur. Laravel ne rend pas de pages HTML PHP ; il expose des routes API JSON. Nginx sert Angular et transmet les appels /api vers Laravel.

1. Formulaire Angular	2. Requete HTTP	3. Middleware Laravel	4. Controleur PHP	5. Tables MySQL	6. Reponse Angular
-----------------------	-----------------	-----------------------	-------------------	-----------------	--------------------

Point important : les controles du frontend ameliorent l'experience utilisateur, mais Laravel revalide toujours les donnees et les permissions. Un utilisateur ne peut donc pas contourner un droit en modifiant le navigateur.

Fichiers centraux : frontend/src/app/app.routes.ts, frontend/src/app/core/auth/auth.service.ts, backend/routes/api.php et backend/app/Http/Controllers/Api/AuthController.php.

2. Pages du frontend Angular

URL	Composant	Fonction visible
/login	LoginComponent	Connexion email/mot de passe ; bouton de connexion par passkey ; lien mot de passe oublie.
/register	RegisterComponent	Creation de compte avec nom, email, mot de passe et acceptation des conditions.
/forgot-password	ForgotPasswordComponent	Envoie une demande de lien de reinitialisation.
/reset-password	ResetPasswordComponent	Accepte le token de lien et enregistre le nouveau mot de passe.
/dashboard	DashboardComponent	Accueil protege ; affiche le prenom et propose Security ou Sign out.
/security	SecurityComponent	Liste et supprime les passkeys ; affiche, revoque les sessions actives.
/admin	AdminComponent	Liste les utilisateurs si le droit users.view est present.
/forbidden	ForbiddenComponent	Affiche le refus d'accès.

Chaque page est maintenant separee en trois fichiers : component.ts (logique), component.html (interface) et component.scss (style).

3. Connexion par mot de passe - aller

Etape 1 - LoginComponent. Le visiteur remplit email et mot de passe. Le formulaire Angular verifie que les champs sont presents. La methode submit() appelle AuthService.login().

Etape 2 - CSRF. AuthService demande d'abord GET /api/auth/csrf-cookie. Le token recu est memorise et auth.interceptor.ts ajoute X-CSRF-TOKEN aux POST, PUT et DELETE.

Etape 3 - Nginx. Le navigateur appelle localhost:4200/api/auth/login. Le fichier frontend/nginx.conf transmet cette requete au conteneur backend:8000/api/auth/login.

Etape 4 - route PHP. backend/routes/api.php associe POST auth/login a AuthController::login() et applique web, auth.audit et throttle:5,1.

Etape 5 - AuthController::login(). Cette fonction valide les champs, cherche le compte et refuse les identifiants faux, les comptes inactifs et les emails non verifiés.

Etape 6 - MySQL. Laravel lit users.email, users.password, users.is_active et users.email_verified_at. password_verify() compare le mot de passe reçu au hash stocké.

Extrait simplifié du contrôleur :

```
$user = User::where('email', strtolower($credentials['email']))->first();
if (! $user || ! $user->is_active || ! password_verify($credentials['password'], $user->password)) {
    return response()->json(..., 401); }
if (! $user->hasVerifiedEmail()) return response()->json(..., 403);
```

4. Connexion par mot de passe - retour

Etape 7 - Tokens. Laravel génère un access token JWT court et un refresh token plus long.

AuthSessionManager::record() inscrit une session dans auth_sessions avec le hash du jti du refresh token, l'appareil, l'IP et user-agent.

Etape 8 - Reponse. tokenResponse() retourne access_token et user en JSON. Le refresh token est placé dans le cookie refresh_token, HttpOnly ; JavaScript ne peut pas le lire.

Etape 9 - Angular. AuthService.applyAuthentication() conserve access_token et currentUser en mémoire. LoginComponent redirige vers /dashboard.

Etape 10 - Requetes protegees. auth.interceptor.ts ajoute Authorization: Bearer access_token. Laravel applique auth:api puis access.token, qui refuse explicitement un refresh token utilisé comme access token.

Reponse JSON	Access token en memoire	Cookie HttpOnly	Redirection Dashboard
--------------	-------------------------	-----------------	-----------------------

Expiration : si une API retourne 401, l'intercepteur appelle POST /api/auth/refresh. Laravel vérifie le cookie, la session active et le compte, révoque l'ancien refresh token puis en génère une nouvelle paire. Angular rejoue ensuite la requête initiale.

5. Inscription et vérification email

RegisterComponent -> POST /api/auth/register -> AuthController::register(). Laravel impose le prénom, nom, email unique, mot de passe d'au moins 12 caractères avec majuscule, minuscule, chiffre et caractère spécial, puis accepte les conditions.

Dans une transaction SQL, User::create() insère le compte dans users et roles()->attach() associe le rôle user dans role_user. Ensuite sendEmailVerificationNotification() envoie le message de vérification.

Le lien de vérification arrive sur GET /api/auth/email/verify/{id}/{hash}. verifyEmail() contrôle la signature de route et le hash de l'email, puis remplit users.email_verified_at. Sans cette date, login() refuse la connexion avec 403.

Attention email : avec MAIL_MAILER=log, les liens sont écrits dans storage/logs/laravel.log. Pour une réception réelle, la configuration SMTP doit être renseignée dans .env.

6. Mot de passe oublié et réinitialisation

Demande. ForgotPasswordComponent appelle POST /api/auth/password/forgot. AuthController::forgotPassword() utilise Password::sendResetLink(). Laravel crée un token temporaire hashé dans password_reset_tokens et envoie le lien.

La reponse est toujours similaire, que l email existe ou non : cela empeche un attaquant de decouvrir les comptes existants.

Redirection. Le lien email pointe vers GET /api/auth/password/reset/{token}. passwordResetPage() redirige vers Angular : /reset-password?token=...&email;=...

Nouveau mot de passe. ResetPasswordComponent lit token et email dans l URL, puis appelle POST /api/auth/password/reset. AuthController::resetPassword() demande au Password Broker de verifier le token ; si valide, il met a jour users.password et invalide le token.

Forgot Password UI	password_reset_tokens	Email de lien	Reset Password UI	users.password mis a jour
--------------------	-----------------------	---------------	-------------------	---------------------------

7. Passkeys et sessions

SecurityComponent. La page charge GET /api/passkeys et GET /api/sessions. PasskeyController::index() lit passkeys de l utilisateur ; SessionController::index() retourne les auth_sessions non revokees.

Ajouter une passkey. SecurityComponent appelle Passkeys.register(). Le navigateur ouvre Windows Hello, empreinte, Face ID ou une cle physique. Laravel valide WebAuthn puis stocke credential_id et credential dans passkeys.

Se connecter par passkey. LoginComponent appelle Passkeys.verify(). Apres validation, PasskeyLoginResponse cree les memes access et refresh tokens qu une connexion mot de passe. Angular appelle restoreSession(), puis GET /api/auth/me.

Revoquer. DELETE /api/sessions/{id}, DELETE /api/sessions ou DELETE /api/passkeys/{id} sont controles par permission et par la propriete de la ressource : un utilisateur ne peut modifier que ses propres sessions et passkeys.

8. Roles, permissions et audit

Element	Role dans la securite
roles / role_user	Definit les roles attribues au compte : user, admin, etc.
permissions / permission_role	Associe les autorisations fines aux roles, par exemple users.view ou sessions.revoke.
PermissionMiddleware	Bloque les routes admin, sessions et passkeys sans le droit necessaire ; reponse 403.
audit_logs	AuthenticationAuditMiddleware garde une trace des inscriptions, connexions, echecs, deconnexions et resets.

La page Admin n est donc pas seulement cachee dans Angular. Son appel GET /api/admin/users passe aussi par auth:api, access.token et permission:users.view dans Laravel.

9. Carte des fichiers essentiels

Zone	Fichier	Responsabilite
Frontend	core/auth/auth.service.ts	CSRF, login, refresh, logout, sessions et etat utilisateur.
Frontend	core/auth/auth.interceptor.ts	Ajoute Bearer et CSRF ; tente refresh apres un 401.

Zone	Fichier	Responsabilite
Frontend	features/auth/*	HTML, CSS et TypeScript des ecrans auth.
Proxy	frontend/nginx.conf	Transmet /api et les routes passkeys au backend.
Backend	routes/api.php	Declare les URL, limites de debit et middleware.
Backend	Api/AuthController.php	Register, login, refresh, forgot, reset, verify, resend, logout, me.
Backend	Services/AuthSessionManager.php	Enregistre, verifie et revoque les sessions de refresh.
Data	database/migrations/*	Structure de users, tokens reset, sessions, roles, permissions, passkeys et audit.

Conclusion : Angular collecte et affiche, Nginx achemine, Laravel valide et autorise, MySQL conserve les donnees. Les tokens et les controles serveur ferment la boucle vers une interface protegee.